🟢 1. useState – React Hook

🔧 SYNTAX

```
const [order, setOrder] = useState(initialValue);
```

- `order` → current value
- `setOrder` → function to update value
- `initialValue` → the default starting value

📦 CODE EXAMPLE

```
// 📥 Import useState
import React, { useState } from 'react';

// ⚙️ Functional Component <Order>
function Order() {
  // Initial value: 0
  const [order, setOrder] = useState(0);

  return (
    <div>
      <h1>Order: {order}</h1>

      <button onClick={() => setOrder(order + 1)}>Add</button>
      <button onClick={() => setOrder(order - 1)}>Minus</button>
      <button onClick={() => setOrder(0)}>Reset</button>
    </div>
  );
}

export default Order;
```

✅ **Note:** `setOrder(Order + 1)` should be `setOrder(order + 1)` (JavaScript is case-sensitive)

🧠 Summary

- useState lets you manage **reactive state**
- Updating the state using `setOrder()` will **re-render** the component
- Perfect for counters, forms, toggles, and input tracking

🔵 2. useEffect – React Hook

🔧 SYNTAX

```
useEffect(() => {
  // Side-effect logic here
}, [dependencies]);
```

- Runs after component renders
- Re-runs when values in `[dependencies]` change
- If `[]` is empty → runs **only once** (on mount)

🧠 EXPLAIN

- `useEffect` is used to handle **side effects** like:
  - ✅ API calls
  - ✅ Timers
  - ✅ Event listeners
  - ✅ DOM updates
- Keeps your UI and logic **in sync** with outside systems

🍽️ Real-Life Analogy: Hotel Order Example

Imagine you're in a hotel:

- A **customer** enters (component loads)
- A **waiter** (useEffect) notices
- He takes the **order** (state)
- Then runs to the **kitchen** (API call or timer) to prepare the food
- If **order changes**, he goes again
- When the customer leaves, the **table is cleaned** (cleanup function)

🧠 `useEffect` = Waiter who watches and reacts.

📦 CODE EXAMPLE

```
// 📥 Import useEffect & useState
import React, { useState, useEffect } from 'react';

function UserData() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    // 🧙‍♂️ Side effect: fetch user data from API
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data));
  }, []); // Empty array = run only once on mount

  return (
    <div>
      <h2>👥 User List</h2>
      {users.map(user => (
        <p key={user.id}>{user.name}</p>
      ))}
    </div>
  );
}

export default UserData;
```

🧽 CLEANUP EXAMPLE

```
useEffect(() => {
  const interval = setInterval(() => {
    console.log("⏱️ Tracking...");
  }, 1000);

  return () => clearInterval(interval); // 🔁 Cleanup when component unmounts
}, []);
```

🧠 Summary

Concept	Meaning
Side Effect	Something that affects outside the UI
Empty `[]`	Runs once when component mounts
`[value]`	Runs when `value` changes
Return function	Used for cleanup (like timers/events)

🧠 `useEffect` = “Do this after render, and maybe again if something changes.”

🟠 3. useContext – React Hook

🔧 SYNTAX

```
const value = useContext(MyContext);
```

- Used to **access shared global data**
- Avoids **prop drilling** (no need to pass data through every level)
- Requires a **Context Provider**

🧠 EXPLAIN

- `useContext` lets multiple components **access the same data** without passing it manually
- Ideal for **themes, authentication, language settings, user data** etc.

📶 Real-Life Analogy: The Wi-Fi Board Example

Imagine you're running a company.

- The **boss** writes the Wi-Fi password on a whiteboard
- Every **employee** just reads from the board
- No one asks the boss again and again!

💡 That's `useContext` !

⚙️ HOW TO USE

✅ 1. Create the Context

```
import { createContext } from 'react';

export const WifiContext = createContext();
```

✅ 2. Wrap Components with Provider

```
import React, { useState } from 'react';
import { WifiContext } from './WifiContext';
import App from './App';

function WifiProvider() {
  const [wifiPassword] = useState("Coffee@123");

  return (
    <WifiContext.Provider value={wifiPassword}>
      <App />
    </WifiContext.Provider>
  );
}

export default WifiProvider;
```

✅ 3. Access in Any Child Component

```
import React, { useContext } from 'react';
import { WifiContext } from './WifiContext';

function Employee() {
  const wifi = useContext(WifiContext); // 🧠 Getting value from board

  return (
    <div>
      <h3>📶 Wi-Fi Password: {wifi}</h3>
    </div>
  );
}

export default Employee;
```

🧠 Summary

Concept	Meaning
`createContext()`	Creates shared global context
`.Provider`	Sends the value (like boss writing on board)
`useContext()`	Reads the value anywhere (like employee reading)
Use Case	Theme, auth, language, global user data

🧠 `useContext` = Global whiteboard.
One person writes, everyone can read. No need to pass papers down the line!

🟣 4. useRef – React Hook

🔧 SYNTAX

```
const ref = useRef(initialValue);
```

- Stores a **mutable reference** (like a box)
- Does **not cause re-render** when updated
- Commonly used for:
  - ✅ Accessing DOM nodes
  - ✅ Storing timers
  - ✅ Holding previous values

🧠 EXPLAIN

- `useRef` is like a **secret pocket** inside your component.
- You can store **anything**, and React won't re-render when it changes.
- It's often used to **directly control a DOM element** (like focusing input).

🧮 Real-Life Analogy: The Calculator-in-Hand Example

Imagine you're a **cashier**.

- You keep a **calculator in your hand**
- You don't tell the manager every time you use it (no re-render)
- You use it when needed (DOM access), but it's **not part of your official report**

💡 That's `useRef`.

📦 CODE EXAMPLE – Focusing an Input

```
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null); // 🟪 Like holding calculator

  const handleFocus = () => {
    inputRef.current.focus(); // ⬅️ Direct DOM access
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Enter your name" />
      <button onClick={handleFocus}>🎯 Focus Input</button>
    </div>
  );
}

export default FocusInput;
```

🔁 CODE EXAMPLE – Store a Timer ID

```
const timerRef = useRef();

useEffect(() => {
  timerRef.current = setInterval(() => {
    console.log("⏱️ Tick");
  }, 1000);

  return () => clearInterval(timerRef.current); // Clean up
}, []);
```

🧠 Summary

Concept	Meaning
`useRef()`	Stores a mutable reference
`.current`	The actual stored value
DOM access	`ref.current.focus()`
No re-render	Changing `.current` doesn't trigger re-render
Use case	DOM, timers, previous values, imperatives

🧠 `useRef` = "Calculator in your hand — helps you act fast, but doesn't change the report (UI)."

🟡 5. useMemo – React Hook

🔧 SYNTAX

```
const memoizedValue = useMemo(() => computeExpensiveValue(dep), [dep]);
```

- Runs the function **only when dependencies change**
- ✅ Caches (remembers) the result
- 🚫 Prevents **unnecessary recalculations**

🧠 EXPLAIN

- `useMemo` is used to **optimize performance**
- Perfect when you have **heavy calculations**
- It **skips re-execution** if input hasn't changed

🧾 Real-Life Analogy: Restaurant Bill Calculator

Imagine you're the **cashier** at a hotel.

- A group orders 20+ items
- You calculate the full bill (takes time)
- If they **add one more** item, you recalculate
- But if **nothing changed**, you just reuse the old total!

💡 That's `useMemo` — **calculate only when needed**.

📦 CODE EXAMPLE – Expensive Calculation

```
import React, { useMemo, useState } from 'react';

function BillCalculator() {
  const [items, setItems] = useState([100, 200, 150]);
  const [tax, setTax] = useState(0.18);

  const total = useMemo(() => {
    console.log("💰 Calculating total...");
    return items.reduce((sum, price) => sum + price, 0) * (1 + tax);
  }, [items, tax]); // ⬅️ Only recalculate if items or tax change

  return (
    <div>
      <h3>🧾 Total Bill: ₹{total.toFixed(2)}</h3>
    </div>
  );
}

export default BillCalculator;
```

🧠 Summary

Concept	Meaning
useMemo()	Caches a value based on dependencies
Runs only when…	Dependencies change
Best use case	Expensive calculations, filtering, sorting etc.
Return value	Reused on future renders unless changed

🧠 `useMemo` = Like saving your restaurant bill total and **only rechecking if someone changes the order.**

🟤 6. useCallback – React Hook

🔧 SYNTAX

```
const memoizedFn = useCallback(() => {
  // callback logic here
}, [dependencies]);
```

- Returns a **memoized version of a callback function**
- Only **recreates the function** if dependencies change
- Used to prevent **unnecessary re-renders** or **function re-creation**

🧠 EXPLAIN

- In React, functions are **recreated on every render** by default

- `useCallback` helps to **avoid passing a new function each time**
- Useful when you pass functions to **child components**, and want to avoid unnecessary re-renders

🧉 Real-Life Analogy: Coffee Button in a Hotel

Imagine you're in a hotel room with a **coffee machine**.

- You press the 🧉 **"Make Coffee"** button (function)
- Hotel staff don't need to reset it every second — unless you change **coffee powder** or **water**
- If nothing changed, the same button works again!

💡 That's `useCallback` — it **reuses the function**, unless something it depends on changes.

📦 CODE EXAMPLE – Button Callback with Memoization

```jsx
import React, { useState, useCallback } from 'react';

function CoffeeOrder() {
  const [count, setCount] = useState(0);
  const [orderNote, setOrderNote] = useState("");

  const handleOrder = useCallback(() => {
    console.log("☕ Order placed with note:", orderNote);
  }, [orderNote]); // Only recreates when orderNote changes

  return (
    <div>
      <input
        type="text"
        placeholder="Add order note"
        onChange={(e) => setOrderNote(e.target.value)}
      />
      <button onClick={handleOrder}>Place Order</button>
      <p>Orders placed: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}

export default CoffeeOrder;
```

⚠️ When to Use?

- Passing **callbacks to optimized child components** (`React.memo`)
- Inside custom hooks
- When working with `useEffect` or event listeners

🧠 Summary

Concept	Meaning
useCallback()	Memoizes a function
Avoids re-creation	Only changes if dependencies change
Use case	Pass function to child without triggering re-render
Compare with	`useMemo` for values, `useCallback` for functions

🧠 `useCallback` = Reuse your coffee button.
Don't remake it unless the ingredients (dependencies) change ☕

🧵 7. useReducer – React Hook (State Manager)

🔧 SYNTAX

```
const [state, dispatch] = useReducer(reducerFunction, initialState);
```

- `state` → current state value
- `dispatch(action)` → used to update state
- `reducerFunction(state, action)` → function that returns the new state
- `initialState` → starting state

🧠 EXPLAIN

- `useReducer` is like **a mini Redux** inside a component
- Useful when:
  - ✅ State logic is **complex**
  - ✅ You have **multiple sub-states**
  - ✅ Updates depend on **previous state**
- Cleaner alternative to `useState` for complex logic

🍽️ Real-Life Analogy: Hotel Order Management

Imagine you run a hotel kitchen:

- You have a **notebook** (state) for order counts
- The waiter gives you **instructions** like "Add 1", "Reset", "Remove 1"
- Based on the instruction (`action.type`), you **update the notebook**

💡 That's `useReducer` — a central logic to handle many actions.

📦 CODE EXAMPLE – Order Counter

```jsx
import React, { useReducer } from 'react';

// Reducer function – handles all actions
function orderReducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return { count: 0 };
    default:
      return state;
  }
}

const initialState = { count: 0 };

function HotelOrder() {
  const [state, dispatch] = useReducer(orderReducer, initialState);

  return (
    <div>
      <h2>🍽️ Orders: {state.count}</h2>

      <button onClick={() => dispatch({ type: 'increment' })}>Add Order</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>Cancel Order</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}

export default HotelOrder;
```

🧠 Summary

Concept	Meaning
`useReducer()`	Hook for managing complex state
`state`	Current value of the state
`dispatch()`	Function to send actions
`reducer()`	Pure function to update state based on action
Use case	Forms, toggles, carts, auth, multi-step logic

🧠 `useReducer` = Hotel notebook for managing different order instructions with one central rulebook (the reducer)!

🔐 8. useImperativeHandle – React Hook (Custom Ref Control)

🔧 SYNTAX

```
useImperativeHandle(ref, () => ({
  method1,
  method2,
}), [dependencies]);
```

- Used with `forwardRef`
- Lets you **expose specific functions or values to parent**
- Parent can **call methods inside the child** like `.focus()`, `.reset()`, etc.

🧠 EXPLAIN

Normally, `ref` is used to access DOM nodes.
But sometimes you want to **expose custom functions from a child component to its parent**.

That's what `useImperativeHandle` does:

- Gives the parent **controlled access** to child functions
- Used rarely, but powerful in cases like:
  - Input focus
  - Form reset
  - Custom animations
  - Modal open/close

🔔 Real-Life Analogy: Hotel Room Service Bell

Imagine you're staying in a hotel.

- You don’t go to the kitchen (child) every time.
- You just press the **service bell** from your room (ref).
- Only certain actions are allowed: "Bring Water", "Make Coffee".

💡 That’s `useImperativeHandle` — the **service bell** gives parent access to a few allowed actions inside the child component.

📦 CODE EXAMPLE – Focus Input from Parent

✅ Child: InputBox.jsx

```
import React, { useRef, forwardRef, useImperativeHandle } from 'react';

const InputBox = forwardRef((props, ref) => {
  const inputRef = useRef();

  useImperativeHandle(ref, () => ({
    focusInput() {
      inputRef.current.focus(); // Exposed method
    },
    clearInput() {
      inputRef.current.value = '';
    }
  }));

  return <input ref={inputRef} placeholder="Type something..." />;
});

export default InputBox;
```

✅ Parent: App.jsx

```jsx
import React, { useRef } from 'react';
import InputBox from './InputBox';

function App() {
  const inputBoxRef = useRef();

  return (
    <div>
      <InputBox ref={inputBoxRef} />
      <button onClick={() => inputBoxRef.current.focusInput()}>Focus Input</button>
      <button onClick={() => inputBoxRef.current.clearInput()}>Clear Input</button>
    </div>
  );
}

export default App;
```

🧠 Summary

Concept	Meaning
`useImperativeHandle`	Customize what a parent can access via `ref`
`forwardRef()`	Required to forward the ref to child
`.focus()`, `.clear()`	Examples of methods exposed to parent
Use cases	Form control, modals, animations, focus handling

🧠 `useImperativeHandle` = Like pressing a hotel bell that only triggers **approved services** from the child (input, modal, etc.)

🧩 9. useLayoutEffect – React Hook (Sync DOM Effect)

🔧 SYNTAX

```
useLayoutEffect(() => {
  // sync DOM effect here
}, [dependencies]);
```

- Runs **after the DOM is updated**, but **before the browser paints**
- Runs synchronously → **blocks painting**
- Use only when you **must read layout or measure DOM**

🧠 EXPLAIN

- Very similar to `useEffect`, but **executes earlier**
- Use when:
  - You need to **measure DOM elements**
  - Adjust styles before screen shows
  - Fix UI shifts (prevent flicker)

🏨 Real-Life Analogy: Adjusting Table Setup Before Guests See It

Imagine you're in a restaurant:

- A waiter quickly aligns the tablecloth straight **before** the customer walks in.
- If he adjusts it **after** the customer sees it → awkward!

💡 That’s `useLayoutEffect` .

Used when you want to **do something before the screen shows it** to the user.

📦 CODE EXAMPLE – Auto Scroll Before Paint

```jsx
import React, { useLayoutEffect, useRef } from 'react';

function ScrollBox() {
  const boxRef = useRef();

  useLayoutEffect(() => {
    // Scroll to bottom before screen is painted
    boxRef.current.scrollTop = boxRef.current.scrollHeight;
  }, []);

  return (
    <div
      ref={boxRef}
      style={{
        height: "150px",
        width: "300px",
        overflow: "auto",
        border: "1px solid gray",
      }}
    >
      <p>Item 1</p>
      <p>Item 2</p>
      <p>Item 3</p>
      <p>Item 4</p>
      <p>Item 5</p>
      <p>Item 6</p>
      <p>Item 7</p>
      <p>Item 8</p>
      <p>Item 9</p>
      <p>Item 10</p>
    </div>
  );
}

export default ScrollBox;
```

⚠️ Warning

- **Blocks paint** → avoid heavy logic inside
- Prefer `useEffect` unless you see layout glitches or need measurements
- Use only when **timing is critical**

🧠 Summary

Concept	Meaning
`useLayoutEffect`	Runs **before paint**, **after DOM update**
Synchronous?	✅ Yes
Use when?	Measuring layout, fixing flicker, sync position
Caution	Don't use for heavy tasks – can delay rendering

🧠 `useLayoutEffect` = “Fix the table setup quickly before guests (users) walk in and see it!”

🆔 10. useId – React Hook (Unique ID Generator)

🔧 SYNTAX

```
const id = useId();
```

- Generates a **unique and stable ID string** for this component instance
- Same across client/server — important for SSR (Server-Side Rendering)

- Useful for inputs, labels, modals, etc.

🧠 EXPLAIN

- Every time a component is created, `useId()` gives it a **unique ID**
- You can **append** strings to make it readable or assign it to form elements
- Prevents **ID collisions** in the DOM (duplicate IDs = ❌ bad for accessibility)

🏨 Real-Life Analogy: Room Numbers in a Hotel

Imagine a hotel assigns room numbers.

- Every guest gets a **unique room number**
- Two guests can't have **room 101** — that causes confusion!
- So React gives each guest (component) their **own stable ID**

💡 That's `useId` — unique room numbers for each element that need to be identified.

📦 CODE EXAMPLE – Input & Label

```
import React, { useId } from 'react';

function NameInput() {
  const id = useId(); // 🆔 Unique ID

  return (
    <div>
      <label htmlFor={id}>Name:</label>
      <input id={id} type="text" placeholder="Enter your name" />
    </div>
  );
}

export default NameInput;
```

🔄 CODE EXAMPLE – Multiple IDs in a List

```
import React, { useId } from 'react';

function MenuList() {
  const id = useId();

  const items = ["Coffee", "Tea", "Juice"];
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index} id={`${id}-${index}`}>
          {item}
        </li>
      ))}
    </ul>
  );
}
```

🧠 Summary

Concept	Meaning
`useId()`	Generates unique, stable IDs for accessibility
SSR safe?	✅ Yes – same on server and client
Use case	Forms, labels, modals, lists
Format	Looks like: `:r1:` , `:r2:` , `:r1-0`

🧠 `useId` = Give each guest (component) their own **room number**, so no one gets mixed up in the hotel 🏨

🧪 11. use – React 19 Hook (Handle Async Promises Directly)

🔧 SYNTAX

```
const data = use(promise);
```

- Accepts a **Promise**
- Suspends the component until the promise resolves
- Can be used inside **Server Components** and **Client Components** (with Suspense)
- **No need for** `useEffect`, `useState`, or even `async/await` in many cases!

🧠 EXPLAIN

- `use()` lets you **await promises directly** in React component body
- React **pauses rendering** until the promise resolves
- Helps **fetch data cleanly** without boilerplate

🍛 Real-Life Analogy: Order, Wait & Serve

Imagine you're at a restaurant.

- You order a biryani (promise)
- Waiter tells you: "Please wait until it's ready" (suspense)
- Once biryani is ready → waiter serves it (resolved promise)

💡 That's `use()` → it **waits silently**, and once data is ready, React continues rendering.

📦 CODE EXAMPLE – Fetch Todo

> ✅ Works in components wrapped with `<Suspense>`

```
import React, { Suspense, use } from 'react';

const todoPromise = fetch("https://jsonplaceholder.typicode.com/todos/1")
  .then(res => res.json());

function Todo() {
  const todo = use(todoPromise); // ⏳ Suspends until resolved

  return (
    <div>
      <h2>✅ {todo.title}</h2>
    </div>
  );
}

export default function App() {
  return (
    <Suspense fallback={<p>Loading todo...</p>}>
      <Todo />
    </Suspense>
  );
}
```

🔁 CODE EXAMPLE – With Async Function

```
function fetchUser() {
  return fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(res => res.json());
}

const userPromise = fetchUser();

function Profile() {
  const user = use(userPromise);

  return <h1>Hello, {user.name}</h1>;
}
```

🧠 Summary

Concept	Meaning
`use()`	Handles promises directly inside component
Suspense?	✅ Required to wrap component
Replaces	`useEffect + useState` combo for async data
Works in	React 19, with async-supported environments

🧠 `use()` = “Place the order, wait silently behind the scenes, then React continues once it's served.”

🌟 React 19 Form & UX Hooks

🥇 12. useOptimistic – Instant UI Before Server Confirms

🔧 SYNTAX

```
const [optimisticValue, addOptimistic] = useOptimistic(value, updaterFn);
```

- `value` : actual source of truth
- `addOptimistic(input, fn)` : apply optimistic update (temporary)
- UI **instantly shows result** before server confirms

💡 Real-Life Analogy: Instant Bill on POS Machine

Imagine ordering food:

- You pay → receipt prints instantly (optimistic)
- Actual bill from kitchen comes 5 secs later (real confirmation)

💡 `useOptimistic` = print receipt **before** confirmation

📦 CODE EXAMPLE

```
'use client';
import { useState, useOptimistic } from 'react';

function Comments() {
  const [comments, setComments] = useState([]);
  const [optimisticComments, addOptimisticComment] = useOptimistic(
    comments,
    (current, newComment) => [...current, newComment]
  );

  const handleSubmit = async (formData) => {
    const comment = formData.get("comment");
    addOptimisticComment({ text: comment });

    await new Promise((res) => setTimeout(res, 1000)); // simulate server
    setComments(prev => [...prev, { text: comment }]);
  };

  return (
    <form action={handleSubmit}>
      <input name="comment" />
      <button type="submit">Post</button>
      <ul>
        {optimisticComments.map((c, i) => <li key={i}>{c.text}</li>)}
      </ul>
    </form>
  );
}
```

🥈 13. useFormStatus – Get Form Submit Status (Inside Form)

🔧 SYNTAX

```
const { pending, action } = useFormStatus();
```

- Tells if the **form is currently submitting**
- Used inside a `form` to **disable buttons**, show loading, etc.

💡 Real-Life Analogy: Waiter Sending Order to Kitchen

- While waiter is sending your order (form submitting), you can't place another order
- Once done → order is enabled again

💡 `useFormStatus` = Track whether the form is **in progress**

📦 CODE EXAMPLE

```
'use client';
import { useFormStatus } from 'react-dom';

function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button type="submit" disabled={pending}>
      {pending ? "Sending..." : "Submit"}
    </button>
  );
}

function ContactForm() {
  async function handleSubmit(formData) {
    await new Promise(res => setTimeout(res, 2000)); // simulate delay
  }

  return (
    <form action={handleSubmit}>
      <input name="email" placeholder="Your email" />
      <SubmitButton />
    </form>
  );
}
```

🥉 14. useFormState – Manage Server Form State in Client

🔧 SYNTAX

```
const [state, formAction] = useFormState(actionFn, initialState);
```

- Helps manage **form state returned from server**

- Works with **Server Actions** (like validation result or status)

💡 Real-Life Analogy: OTP Verification

- You submit OTP form
- Server replies: ✅ Valid or ❌ Wrong
- Form remembers the result and shows message

💡 `useFormState` = Manage **form responses from server** easily

📦 CODE EXAMPLE

```
'use client';
import { useFormState } from 'react-dom';

async function verifyOtp(prevState, formData) {
  const otp = formData.get("otp");
  if (otp === "1234") return "✅ Verified";
  return "❌ Incorrect OTP";
}

function OtpForm() {
  const [message, formAction] = useFormState(verifyOtp, null);

  return (
    <form action={formAction}>
      <input name="otp" />
      <button type="submit">Verify</button>
      {message && <p>{message}</p>}
    </form>
  );
}
```

🧠 Summary Table

Hook	Purpose
`useOptimistic`	Show UI changes **instantly**, update later
`useFormStatus`	Know if form is **submitting** (loading)
`useFormState`	Manage **server-side state** of a form

💡 React 19 makes forms feel **native, fast, and smooth**, giving full control to both **UI and server interaction**.